

Wheeled Biped Robot (RoTino)

Manuale Tecnico Completo:
Architettura del Codice e Fondamenti Teorici
Analisi file per file con collegamento alla teoria del controllo

Analisi Tecnica Approfondita

October 1, 2026

Indice

1	Introduzione e Architettura del Progetto	2
1.1	Obiettivo del Documento	2
1.2	Albero Completo dei File	2
2	rotino_description: Il Modello Dinamico del Robot	4
2.1	Ruolo del Pacchetto	4
2.2	File: kinematics.py	4
2.2.1	Ruolo	4
2.2.2	Riga 9–17: Conversione RPY a matrice di rotazione	4
2.2.3	Riga 20–24: Formula di Rodrigues (Asse-Angolo)	5
2.2.4	Riga 54–71: Cinematica Diretta ad Albero	5
2.3	File: model.py	6
2.3.1	Ruolo	6
2.3.2	Riga 28–48: Struttura dati WBRParams (Tabella 1 del Paper)	6
2.3.3	Riga 151–178: Centroidi Equivalente (Eqs. 2-3 del Paper)	6
2.3.4	Riga 180–195: Coefficienti VL-WIP (Eq. 14 del Paper)	7
2.3.5	Riga 211–223: FK Analitica 2D delle Gambe e Jacobiano	7
2.4	File: planar_trajectory.py	8
2.4.1	Ruolo	8
2.4.2	Riga 50–63: Legge Oraria Quintica e Curvatura	8
3	rotino_mpc: Model Predictive Control	10
3.1	Architettura MPC	10
3.2	Classe AxisKalman (solvers.py)	10
3.2.1	Ruolo	10
3.2.2	Righe 136–147: Predizione e Correzione del Filtro di Kalman	10
3.3	File: solvers.py	11
3.3.1	Riga 15–30: CARE e Guadagno LQR Ottimale	11
3.3.2	Riga 33–45: LQRSchedule (TV-LQR su griglia di lunghezze)	11
3.3.3	Riga 48–58: Fast QP con Algoritmo FISTA	12
3.3.4	Riga 61–76: Condensazione del Problema MPC	12
4	rotino_pid: PID con lo Zero Moment Point	14
4.1	Architettura del Controllore	14
4.2	File: zmp_balance.py	14
4.2.1	Righe 86–123: Capture Point, ZMP desiderato e PD sull’offset ZMP	14
4.2.2	Righe 139–153: Anteprima LIPM	15
4.3	File: controller.py (PID)	15
4.3.1	Righe 505–524: PD cartesiano delle gambe	15

5	rotino_benchmark: Il Confronto PID–MPC	16
5.1	Organizzazione	16
5.2	File: campaign.py	16
5.3	File: compare.py	17
5.4	File: logger.py	17
5.5	File: plot.py – I Grafici del Confronto	18
6	Conclusioni	19

Capitolo 1

Introduzione e Architettura del Progetto

1.1 Obiettivo del Documento

Questo manuale ha lo scopo di collegare in modo esplicito e rigoroso ogni singolo file sorgente del progetto RoTino alla teoria del controllo e della robotica da cui trae origine. Per ciascun file viene presentata prima una descrizione del ruolo architetturale, poi le righe di codice piu' significative con la corrispondente derivazione matematica.

1.2 Albero Completo dei File

Di seguito e' riportata la struttura completa del workspace ROS 2 Humble:

```
rotino_ws/
|-- src/
|   |-- rotino_description/
|   |   |-- rotino_description/
|   |   |   |-- model.py          <-- Parametri VL-WIP dall'URDF
|   |   |   |-- kinematics.py     <-- FK analitica e CoM globale
|   |   |   |-- planar_trajectory.py <-- Legge oraria S-curve quintica
|   |   |   '-- zmp.py            <-- ZMP multicorpo ricostruito
|   |   '-- urdf/
|   |       '-- rotino.urdf.xacro <-- Geometria, masse, giunti
|   |
|   |-- rotino_pid/
|   |   |-- rotino_pid/
|   |   |   |-- controller.py     <-- Nodo ROS 2: riferimenti, gambe, salto
|   |   |   '-- zmp_balance.py    <-- Cascata ZMP / Capture Point, anteprima LIPM
|   |   '-- test/                <-- Verifica sul VL-WIP linearizzato
|   |
|   |-- rotino_mpc/
|   |   '-- rotino_mpc/
|   |       |-- controller.py     <-- Nodo ROS 2, loop di controllo
|   |       '-- solvers.py        <-- QP FISTA, LQR, Kalman, condensazione
|   |
|   |-- rotino_dashboard/        <-- GUI, teleop, vista ZMP
|   |
|   '-- rotino_benchmark/
```

```

|      '-- rotino_benchmark/
|      |-- scenarios.py      <-- Catalogo degli 8 scenari
|      |-- campaign.py      <-- Uno scenario con PID e MPC
|      |-- suite.py         <-- Tutti gli scenari + riepilogo
|      |-- logger.py        <-- Logging sincrono ad alta freq.
|      |-- compare.py       <-- Metriche, migliore per metrica
|      |-- plot.py          <-- Grafici PID/MPC sovrapposti
|      '-- zmp_analysis.py   <-- Metriche e figure ZMP
'-- docs/
    '-- Architettura_e_Codice.tex <- Questo documento

```

Capitolo 2

rotino_description: Il Modello Dinamico del Robot

2.1 Ruolo del Pacchetto

Questo pacchetto ha due compiti fondamentali: (1) definire la geometria e le inerzie del robot tramite URDF/Xacro, e (2) fornire a tutti gli altri pacchetti una libreria Python che estrae automaticamente i parametri dinamici dal modello URDF, evitando valori scritti a mano che potrebbero discostarsi dalla simulazione.

2.2 File: kinematics.py

2.2.1 Ruolo

Implementa la **cinematica diretta (Forward Kinematics)** del robot traversando l'albero URDF a partire dalla radice, senza dipendenze da librerie esterne come Pinocchio o MoveIt. Calcola il centro di massa globale tramite media ponderata delle masse.

2.2.2 Riga 9–17: Conversione RPY a matrice di rotazione

Teoria: Una rototraslazione rigida nello spazio 3D si rappresenta con la matrice di rotazione $\mathbf{R} \in SO(3)$. Per la convenzione RPY (Roll, Pitch, Yaw) usata da URDF e ROS, la rotazione composta e' il prodotto: $\mathbf{R} = R_z(\psi)R_y(\phi)R_x(\theta)$, la cui forma esplicita e' la classica matrice di rotazione di Eulero ZYX.

```
1 def rpy_to_matrix(r, p, y):
2     cr, sr = np.cos(r), np.sin(r)
3     cp, sp = np.cos(p), np.sin(p)
4     cy, sy = np.cos(y), np.sin(y)
5     # Prodotto R_z * R_y * R_x in forma esplicita (convenzione ZYX)
6     return np.array([
7         [cy*cp, cy*sp*sr - sy*cr, cy*sp*cr + sy*sr],
8         [sy*cp, sy*sp*sr + cy*cr, sy*sp*cr - cy*sr],
9         [-sp,      cp*sr,      cp*cr],
10    ])
```

Listing 2.1: kinematics.py rr. 9-17: Matrice di rotazione da angoli RPY

Questa funzione viene richiamata ogni volta che si propaga la trasformazione di un giunto fisso dell'URDF, che specifica la sua posa relativa tramite attributo `rpy` nella tag `<origin>`.

2.2.3 Riga 20–24: Formula di Rodrigues (Asse-Angolo)

Teoria (Formula di Rodrigues): Data una rotazione di angolo α attorno all'asse unitario $\hat{\mathbf{k}}$, la matrice di rotazione è:

$$\mathbf{R} = \mathbf{I} + \sin \alpha [\hat{\mathbf{k}}]_{\times} + (1 - \cos \alpha) [\hat{\mathbf{k}}]_{\times}^2$$

dove $[\cdot]_{\times}$ è la matrice antisimmetrica (skew-symmetric) tale che $[\mathbf{k}]_{\times} \mathbf{v} = \mathbf{k} \times \mathbf{v}$.

```
1 def axis_angle_to_matrix(axis, angle):
2     k = np.asarray(axis, dtype=float)
3     k = k / np.linalg.norm(k)
4     # kx e' la matrice skew-symmetric [k]_x
5     kx = np.array([[0.0, -k[2], k[1]],
6                    [k[2], 0.0, -k[0]],
7                    [-k[1], k[0], 0.0]])
8     # Formula di Rodrigues esatta
9     return np.eye(3) + np.sin(angle) * kx + (1.0 - np.cos(angle)) * (kx
10 @ kx)
```

Listing 2.2: kinematics.py rr. 20-24: Formula di Rodrigues

Questa è la formula canonica per costruire la matrice di rotazione $R \in SO(3)$ attorno a un asse arbitrario. Viene usata in `link_frames` per ogni giunto revoluto, dove l'asse è quello specificato dall'URDF e l'angolo è la variabile di giunto q_i .

2.2.4 Riga 54–71: Cinematica Diretta ad Albero

Teoria (FK per strutture ad albero): La cinematica diretta propaga la trasformazione omogenea ${}^0T_n = {}^0T_1 \cdot {}^1T_2 \cdots {}^{n-1}T_n$ dal nodo radice verso le foglie. In forma compatta, per ogni link figlio: $p_{child} = p_{parent} + R_{parent} \cdot p_{joint}$, $R_{child} = R_{parent} \cdot R_{joint} \cdot R_q(q_i)$.

```
1 def link_frames(self, base_pos, base_rot, joint_positions):
2     frames = {}
3     stack = [(self.root, base_rot, base_pos)]
4     while stack:
5         name, rot, pos = stack.pop()
6         frames[name] = (rot, pos)
7         for joint in self.child_joints.get(name, []):
8             origin = joint.origin
9             xyz = np.array(origin.xyz if origin.xyz else [0., 0., 0.])
10            rpy = origin.rpy if origin.rpy else [0., 0., 0.]
11            # p_child = p_parent + R_parent * p_joint_origin
12            child_pos = pos + rot @ xyz
13            # R_child = R_parent * R_joint_origin
14            child_rot = rot @ rpy_to_matrix(*rpy)
15            if joint.type in ('revolute', 'continuous'):
16                # Applica la rotazione del giunto con la formula di
17                # Rodrigues
18                child_rot = child_rot @ axis_angle_to_matrix(
19                    joint.axis, joint_positions.get(joint.name, 0.0))
20            stack.append((joint.child, child_rot, child_pos))
21    return frames
```

Listing 2.3: kinematics.py rr. 54-71: FK iterativa via stack DFS

L'approccio DFS (Depth First Search) tramite stack esplicito è equivalente alla ricorsione ma evita l'overhead dello stack Python per alberi profondi.

2.3 File: model.py

2.3.1 Ruolo

Questo e' il nucleo analitico del progetto. Implementa il modello **VL-WIP (Variable-Length Wheeled Inverted Pendulum)** di Cui et al., *Micromachines* 2022, 13, 747, calcolando automaticamente tutti i parametri dinamici dalla geometria URDF. Ogni calcolo ha un riferimento esplicito all'equazione del paper.

2.3.2 Riga 28–48: Struttura dati WBRParams (Tabella 1 del Paper)

```
1 @dataclass
2 class WBRParams:
3     m_w: float # massa di una ruota [kg] -> Tab.1: m_w
4     I_w: float # inerzia di spin della ruota [kg m2] -> Tab.1: I_w
5     r: float # raggio della ruota [m] -> Tab.1: r
6     d: float # carreggiata [m] -> Tab.1: d
7     m_b: float # massa corpo superiore [kg] -> Tab.1: m_b
8     m_1: float # massa dello stinco [kg] -> Tab.1: m_1
9     m_2: float # massa della coscia [kg] -> Tab.1: m_2
10    l_1: float # lunghezza stinco: ginocchio->ruota -> Tab.1: l_1
11    l_2: float # lunghezza coscia: anca->ginocchio -> Tab.1: l_2
12    L_max: float # lunghezza massima gamba = l1 + l2 -> Tab.1: L
13    mu: float # coeff. di attrito ruota/suolo -> Tab.1: mu
14    leg_damping: float # smorzamento viscoso giunto [Nm s/rad]
```

Listing 2.4: model.py rr. 28-48: Struttura parametri VL-WIP

Questi parametri corrispondono esattamente alla Tabella 1 del paper di riferimento. L'estrazione automatica garantisce coerenza tra simulazione e modello analitico.

2.3.3 Riga 151–178: Centroide Equivalente (Eqs. 2-3 del Paper)

Teoria (Eqs. 2-3): Il WBR viene modellato come un pendolo inverso a lunghezza variabile. Il corpo superiore (gambe + torso) e' sostituito da un punto materiale equivalente di massa m_b :

$$S_C = \frac{\sum_i m_i x_i}{m_b}, \quad Z_C = \frac{\sum_i m_i z_i}{m_b}, \quad l = \sqrt{S_C^2 + Z_C^2}, \quad \theta = \arctan\left(\frac{S_C}{Z_C}\right)$$

dove la somma si estende a tutti i link tranne le ruote, e le coordinate sono nel frame dell'asse ruote.

```
1 def equivalent_centroid(self, hip, knee, pitch=0.0):
2     R = rpy_to_matrix(0.0, pitch, 0.0)
3     frames = self.kin.link_frames(np.zeros(3), R, self.leg_joints(hip,
4     knee))
5     # Punto medio dell'asse ruote
6     axle = 0.5*(frames[LEFT_WHEEL_LINK][1] + frames[RIGHT_WHEEL_LINK
7     ] [1])
8     # CoM ponderato del corpo superiore (tutto tranne le ruote)
9     masses = np.array([_link(self.robot, n).inertial.mass for n in names
10    ])
11    coms = np.array([self._link_com(frames, n) for n in names])
12    com = masses @ coms / masses.sum()
13    # Eq. (2)-(3): distanza S_C, Z_C rispetto all'asse ruote
14    S_C = float(com[0] - axle[0])
15    Z_C = float(com[2] - axle[2])
```

```

13     return {
14         'l':      math.hypot(S_C, Z_C),      # lunghezza del pendolo
equivalente
15         'theta': math.atan2(S_C, Z_C),      # angolo rispetto alla
verticale
16         'I_y':    float(I[1, 1]),           # inerzia di pitch del
corpo sup.
17         'z_b':    float(-axle[2]),          # altezza asse ruote da
terra
18     }

```

Listing 2.5: model.py rr. 151-178: Centroide equivalente (Eqs. 2-3)

2.3.4 Riga 180–195: Coefficienti VL-WIP (Eq. 14 del Paper)

Teoria (Eq. 14): Linearizzando le equazioni di Lagrange del WBR attorno all'equilibrio (robot verticale, ruote che rotolano senza slittamento), si ottiene il modello lineare:

$$\ddot{s} = a_1\theta + b_1(\tau_L + \tau_R), \quad \ddot{\theta} = a_2\theta + b_2(\tau_L + \tau_R)$$

dove $a_2 > 0$ implica l'instabilità del pendolo (autovalore positivo). I coefficienti dipendono dalla lunghezza l del pendolo, variabile con la postura.

```

1 def vlwip_coefficients(self, l, I_y=None, I_z=None):
2     p = self.p
3     if I_y is None:
4         I_y = p.m_b * l * l / 3.0 # approx. del paper: corpo a bastone
5     mb, mw, Iw, r, d = p.m_b, p.m_w, p.I_w, p.r, p.d
6     # Denominatore comune (determinante della matrice di inerzia)
7     den = (2*Iw*(I_y + mb*l*l)
8           + (2*l*l*mb*mw + I_y*(mb + 2*mw))*r*r)
9     return {
10         'a1': -G*l*l*mb*mb*r*r / den,      # accoppiamento s <- theta
11         'a2':  G*l*mb*(2*Iw+(mb+2*mw)*r*r)/den, # instabilita' pendolo
12         'b1':  r*(I_y + l*mb*(l+r)) / den,   # guadagno coppia -> s
13         'b2': -(2*Iw + r*(l*mb+(mb+2*mw)*r))/den, # guadagno coppia ->
theta
14         'b3':  d*r / (2*I_z*r*r + d*d*(Iw+mw*r*r)), # guadagno yaw
15     }

```

Listing 2.6: model.py rr. 180-195: Coefficienti VL-WIP (Eq. 14)

Il fatto che $b_2 < 0$ è fondamentale: nella funzione `_pitch_super_twisting`, la coppia fisica alle ruote si calcola dividendo per b_2 , il che inverte automaticamente il segno del segnale di controllo.

2.3.5 Riga 211–223: FK Analitica 2D delle Gambe e Jacobiano

Teoria (Catena cinematica planare): Per un robot a 2 link planari (anca-ginocchio-ruota), la posizione del centro ruota nel frame base è: $\mathbf{p}_{wheel} = \mathbf{p}_{hip} + R(\phi_{hip})\mathbf{p}_{knee|hip} + R(\phi_{hip} + \phi_{knee})\mathbf{p}_{wheel|knee}$. Il Jacobiano $\mathbf{J} = \partial\mathbf{p}_{wheel}/\partial(\phi_{hip}, \phi_{knee})$ si calcola derivando analiticamente rispetto agli angoli di giunto.

```

1 def leg_fk(self, hip, knee):
2     kx, kz = self.knee_xz # offset ginocchio nel frame dell'anca [m]
3     wx, wz = self.wheel_xz # offset ruota nel frame del ginocchio [m]

```

```

4
5     def rot(q, x, z):
6         c, s = math.cos(q), math.sin(q)
7         # Punto ruotato e sua derivata rispetto all'angolo q
8         return (np.array([x*c + z*s, -x*s + z*c]), # posizione
9                 np.array([-x*s + z*c, -x*c - z*s])) # dp/dq (colonna di
10
11     J)
12
13     pk, dpk = rot(hip, kx, kz) # posizione ginocchio e sua
14     derivata
15     pw, dpw = rot(hip + knee, wx, wz) # posizione ruota e sua
16     derivata
17     # J[:,0] = d(p_ruota)/d(hip): ruotare l'anca muove sia ginocchio che
18     ruota
19     # J[:,1] = d(p_ruota)/d(knee): ruotare il ginocchio muove solo la
20     ruota
21     J = np.column_stack((dpk + dpw, dpw))
22     return self.hip_xz + pk + pw, J

```

Listing 2.7: model.py rr. 211-223: FK 2D e Jacobiano analitico

2.4 File: planar_trajectory.py

2.4.1 Ruolo

Genera la traiettoria planare (S-curve laterale) che il robot segue durante la navigazione in avanzamento e sterzata, con una legge oraria quintica che garantisce accelerazione zero agli estremi (jerk minimizzato).

2.4.2 Riga 50–63: Legge Oraria Quintica e Curvatura

Teoria (Polinomio quintico per profili smooth): Un profilo di posizione $s(t)$ con $s(0) = \dot{s}(0) = \ddot{s}(0) = 0$ e $s(T) = L$, $\dot{s}(T) = \ddot{s}(T) = 0$ (“profilo a S”) si ottiene con il polinomio quintico normalizzato $\tau = t/T$:

$$s(\tau) = L(10\tau^3 - 15\tau^4 + 6\tau^5)$$

La velocità $\dot{s} = L(30\tau^2 - 60\tau^3 + 30\tau^4)/T$ ha profilo trapezoidale arrotondato. La curvatura della traiettoria planare è $\kappa = y''/(1 + y'^2)^{3/2}$ e il yaw_rate desiderato è $\dot{\psi} = \kappa \cdot v$.

```

1 def _time_law(self, t):
2     tau = min(max(t / self.duration, 0.0), 1.0) # normalizzazione
3     [0,1]
4     s = self.length * (10*tau**3 - 15*tau**4 + 6*tau**5) # posizione
5     v = self.length * (30*tau**2 - 60*tau**3 + 30*tau**4) / self.
6     duration # velocità
7     return s, v
8
9 def sample(self, t):
10     s, v = self._time_law(t)
11     # Inversione della curva lunghezza-arco (tabella precalcolata)
12     x = float(np.interp(s, self.s_table, self.x_table))
13     dy = self._dy(x) # derivata della S-curve
14     spaziale
15     heading = math.atan(dy) # angolo tangente alla
16     curva

```

```

13     # Curvatura della curva planare: kappa = y'' / (1 + y'^2)^(3/2)
14     curvature = self._ddy(x) / (1.0 + dy*dy)**1.5
15     # yaw_rate desiderato = curvatura * velocita' lineare
16     return x, float(self._y(x)), heading, v, curvature * v

```

Listing 2.8: planar_trajectory.py rr. 50-63: Legge quintica e campionamento

Capitolo 3

rotino_mpc: Model Predictive Control

3.1 Architettura MPC

Il controllore MPC separa il problema in due QP indipendenti seguendo il modello disaccoppiato di Eq. (9) di Cui et al.: uno orizzontale (posizione e velocità nel piano sagittale) e uno verticale (altezza del bacino e forza verticale alle ruote). Questo disaccoppiamento è possibile grazie all'approssimazione del LIP (Linear Inverted Pendulum).

3.2 Classe AxisKalman (solvers.py)

3.2.1 Ruolo

Implementa, uno per asse mondo, il **Filtro di Kalman lineare a due stati** $[p, v]$ per stimare posizione e velocità del corpo superiore, fondendo le misure dell'accelerometro IMU (aggiornamento veloce) con l'odometria delle ruote combinata alla cinematica delle gambe (correzione lenta e accurata).

3.2.2 Righe 136–147: Predizione e Correzione del Filtro di Kalman

Teoria (Kalman Discreto): Per il sistema $\mathbf{x}_{k+1} = F\mathbf{x}_k + B\mathbf{u}_k + w_k$ con $w_k \sim \mathcal{N}(0, Q)$ e osservazione $y_k = \mathbf{x}_k + v_k$ con $v_k \sim \mathcal{N}(0, R)$, le equazioni ottimali sono:
Predizione: $\hat{\mathbf{x}}^- = F\hat{\mathbf{x}}_{k-1} + B\mathbf{u}_{k-1}$, $P^- = FP_{k-1}F^T + Q$
Correzione: $K = P^-(P^- + R)^{-1}$, $\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + K(y - \hat{\mathbf{x}}^-)$, $P = (I - K)P^-$

```
1 def predict(self, acc, dt):
2     # Matrice di transizione cinematica: x=[p,v], input=accelerazione IMU
3     F = np.array([[1.0, dt], [0.0, 1.0]])
4     Bu = np.array([0.5*dt*dt, dt]) # vettore di ingresso (cinematica newtoniana)
5     # Predizione stato: x^- = F x + B u
6     self.x = F @ self.x + Bu * acc
7     # Propagazione covarianza: P^- = F P F^T + Q
8     # Q = q_acc * Bu Bu^T modella il rumore dell'accelerometro
9     self.P = F @ self.P @ F.T + self.q_acc * np.outer(Bu, Bu) + 1e-9 *
    np.eye(2)
10
11 def correct(self, p_obs, v_obs, noise_scale=1.0):
12     R = self.R_nom * noise_scale # covarianza del rumore di misura
13     S = self.P + R # covarianza dell'innovazione
14     K = self.P @ np.linalg.inv(S) # guadagno ottimale di Kalman
```

```

15     self.x = self.x + K @ (np.array([p_obs, v_obs]) - self.x) #
correzione
16     self.P = (np.eye(2) - K) @ self.P # aggiornamento covarianza

```

Listing 3.1: solvers.py rr. 136-147: Filtro di Kalman (Eqs. 20-21 di Cui 2022)

Il termine $q_{acc} \cdot \mathbf{B}_u \mathbf{B}_u^T$ modella un rumore sull'accelerazione (random walk in accelerazione), il che è fisicamente appropriato per un MEMS IMU che ha un bias lentamente variabile.

3.3 File: solvers.py

3.3.1 Riga 15–30: CARE e Guadagno LQR Ottimale

Teoria (LQR): Il controllore LQR minimizza il costo quadratico $J = \int_0^\infty (x^T Q x + u^T R u) dt$ per il sistema lineare $\dot{x} = Ax + Bu$. La soluzione ottimale è $u^* = -Kx$ con $K = R^{-1}B^T P$ dove P soddisfa la CARE: $A^T P + PA - PBR^{-1}B^T P + Q = 0$. La soluzione si trova dall'invariante stabile dell'hamiltoniano $H = \begin{pmatrix} A & -BR^{-1}B^T \\ -Q & -A^T \end{pmatrix}$.

```

1 def care(A, B, Q, R):
2     n = A.shape[0]
3     Rinv = np.linalg.inv(R)
4     # Costruzione della matrice Hamiltoniana (2n x 2n)
5     H = np.block([[A, -B @ Rinv @ B.T], [-Q, -A.T]])
6     w, V = np.linalg.eig(H)
7     # Autovettori con parte reale negativa (sottospazio stabile)
8     stabile = V[:, w.real < 0.0]
9     # Soluzione di Riccati: P = Im_lower * Im_upper^{-1}
10    P = np.real(stabile[n:, :] @ np.linalg.inv(stabile[:n, :]))
11    return 0.5 * (P + P.T) # simmetrizzazione numerica
12
13 def lqr_gain(A, B, Q, R):
14     P = care(A, B, Q, R)
15     return np.linalg.solve(R, B.T @ P) # K = R^{-1} B^T P

```

Listing 3.2: solvers.py rr. 15-30: CARE e guadagno LQR

3.3.2 Riga 33–45: LQRSchedule (TV-LQR su griglia di lunghezze)

Teoria (Time-Varying LQR): Il guadagno LQR ottimale dipende dalla lunghezza l del pendolo, che varia con la postura delle gambe. Anziché risolvere la CARE a ogni passo (computazionalmente costoso), si precalcola il guadagno su una griglia di valori di l e si interpola linearmente a runtime, realizzando un TV-LQR efficiente.

```

1 class LQRSchedule:
2     def __init__(self, model, samples, Q, R):
3         self.l_grid = np.array([s[0] for s in samples])
4         # Precalcolo offline del guadagno ottimale per ogni l nella
griglia
5         self.K_grid = np.array([lqr_gain(*model.vlwip_matrices(l, I_y=iy
), Q, R)
6                                     for l, iy in samples])
7
8     def gain(self, l):
9         l = float(np.clip(l, self.l_grid[0], self.l_grid[-1]))

```

```

10     i = int(np.clip(np.searchsorted(self.l_grid, l)-1, 0, len(self.
11         l_grid)-2))
12     # Interpolazione lineare del guadagno tra due nodi della griglia
13     a = (1 - self.l_grid[i]) / (self.l_grid[i+1] - self.l_grid[i])
14     return (1.0 - a) * self.K_grid[i] + a * self.K_grid[i+1]

```

Listing 3.3: solvers.py rr. 33-45: LQRSchedule con interpolazione su griglia

3.3.3 Riga 48–58: Fast QP con Algoritmo FISTA

Teoria (FISTA): Per QP con vincoli box $\min_u \frac{1}{2}u^T H u + g^T u$ s.t. $l_o \leq u \leq h_i$, il metodo FISTA (Beck & Teboulle 2009) applica discesa del gradiente proiettato con momento di Nesterov:

$$x_{k+1} = \Pi_{[l_o, h_i]} \left(y_k - \frac{H y_k + g}{L} \right), \quad y_{k+1} = x_{k+1} + \frac{t_k - 1}{t_{k+1}} (x_{k+1} - x_k)$$

con $L = \lambda_{\max}(H)$ e $t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}$. La convergenza e' $O(1/k^2)$ anziche' $O(1/k)$ del gradient descent.

```

1 def box_qp(H, g, lo, hi, x0=None, iters=60):
2     # Stima del passo: 1/L dove L = lambda_max(H)
3     L = float(np.linalg.eigvalsh(H)[-1])
4     x = np.clip(np.zeros_like(g) if x0 is None else x0, lo, hi)
5     y, t = x.copy(), 1.0
6     for _ in range(iters):
7         # Passo di discesa proiettato sul box [lo, hi]
8         x_new = np.clip(y - (H @ y + g) / L, lo, hi)
9         # Aggiornamento del momento di Nesterov
10        t_new = 0.5 * (1.0 + np.sqrt(1.0 + 4.0 * t * t))
11        y = x_new + ((t - 1.0) / t_new) * (x_new - x)
12        x, t = x_new, t_new
13    return x

```

Listing 3.4: solvers.py rr. 48-58: FISTA per QP con vincoli box

3.3.4 Riga 61–76: Condensazione del Problema MPC

Teoria (Condensazione): Date le dinamiche a spazio di stato $x_{k+1} = A x_k + B u_k + c$, sostituendo ricorsivamente si ottiene:

$$\mathbf{X} = \Phi x_0 + \Gamma \mathbf{U} + \mathbf{C}_{st}$$

dove $\mathbf{X} = [x_1; \dots; x_N]$, $\mathbf{U} = [u_0; \dots; u_{N-1}]$. Le matrici Φ , Γ , $\mathbf{C}_{st}$ si costruiscono iterativamente. Il costo quadratico si riscrive come QP puro in $\mathbf{U}$: $H = \Gamma^T S_{blk} \Gamma + W I$, $g = \Gamma^T S_{blk} (\Phi x_0 + \mathbf{C}_{st} - X^*)$.

```

1 def _condense(A, B, c, N):
2     n, m = B.shape
3     Phi = np.zeros((N*n, n)); Gam = np.zeros((N*n, N*m)); Cst = np.zeros(N*n)
4     Ak = np.eye(n)
5     for i in range(N):
6         Ak = A @ Ak
7         Phi[i*n:(i+1)*n] = Ak

```

```

8         row = i * n
9         prev = Gam[row-n:row] if i > 0 else np.zeros((n, N*m))
10        Gam[row:row+n] = A @ prev
11        Gam[row:row+n, i*m:(i+1)*m] = B
12        Cst[row:row+n] = (A @ Cst[row-n:row] if i > 0 else 0.0) + c
13    return Phi, Gam, Cst
14
15    class UpperBodyMPC:
16        # Disaccoppiamento orizzontale/verticale (Eq. 9 di Cui 2022)
17        def _solve(self, A, B, c, x0, x_ref, S, W, lo, hi, warm):
18            Phi, Gam, Cst = _condense(A, B, c, self.N)
19            Sb = np.kron(np.eye(self.N), S) # matrice di peso a blocchi
20            # Hessiana H e gradiente g del QP condensato
21            H = Gam.T @ Sb @ Gam + W * np.eye(self.N)
22            g = Gam.T @ Sb @ (Phi @ x0 + Cst - x_ref.reshape(-1))
23            warm = np.clip(np.concatenate((warm[1:], warm[-1:])), lo, hi)
24            u = box_qp(H, g, lo, hi, warm)
25            return u, (Phi @ x0 + Gam @ u + Cst).reshape(self.N, -1)
26
27        def solve(self, x_h, ref_h, h, zdd_ref, ds_max, x_v, ref_v, f_min,
28        f_max):
29            dt = self.dt
30            # LIP orizzontale: s_ddot = (g+z_ddot)/h * delta_s
31            Bh = np.array([[0.5*dt*dt],[dt]]) * (G + zdd_ref) / max(h, 1e-3)
32            self.u_h, self.x_h = self._solve(self.Ah, Bh, np.zeros(2),
33            x_h, ref_h, self.S_h, self.W_h, -ds_max,
34            ds_max, self.u_h)
35            # Verticale: z_ddot = F_z/m_b - g
36            Bv = np.array([[0.5*dt*dt],[dt]]) / self.m_b
37            cv = np.array([-0.5*dt*dt*G, -dt*G]) # termine costante della
38            gravita'
39            self.u_v, self.x_v = self._solve(self.Ah, Bv, cv,
40            x_v, ref_v, self.S_v, self.W_v, f_min,
41            f_max, self.u_v)
42            return float(self.u_h[0]), float(self.u_v[0])

```

Listing 3.5: solvers.py rr. 61-114: Condensazione MPC e classe UpperBodyMPC

Capitolo 4

rotino_pid: PID con lo Zero Moment Point

4.1 Architettura del Controllore

Il pacchetto separa la legge di controllo dal nodo ROS:

- `zmp_balance.py`: legge pura (numpy, senza ROS), verificata offline da `test/test_zmp_balance.py` sul VL-WIP linearizzato;
- `controller.py`: nodo a 500 Hz che ricava gli stati dalla ground truth, genera i riferimenti (scenari, traiettoria planare, comandi della dashboard), chiama la legge e comanda gambe e ruote; contiene anche il supervisore del salto.

Descrizione completa e prove in `docs/PID_ZMP.md`.

4.2 File: `zmp_balance.py`

4.2.1 Righe 86–123: Capture Point, ZMP desiderato e PD sull'offset ZMP

Teoria (LIPM e Capture Point): con il modello cart-table $\ddot{c} = \omega^2(c - p)$, $\omega^2 = g/h$, la componente divergente $\xi = c + \dot{c}/\omega$ evolve come $\dot{\xi} = \omega(\xi - p)$. Imponendo $\dot{\xi} = \dot{\xi}_{ref} - k_{\xi}e_{\xi} - k_i \int e_{\xi}$ si ricava lo ZMP che serve; sulle due ruote lo ZMP longitudinale e' il contatto, quindi l'offset $s = c - p$ fra baricentro e asse e' la grandezza che la coppia delle ruote deve portare a s_{des} . L'autorita' $\rho = 0.4$ (authority) tiene conto del fatto che l'anello interno non e' infinitamente veloce.

```
1 omega = math.sqrt(G / max(h, 0.03))
2 s_ff, tau_ff = self.feedforward(acc_ref)           # lean and torque of
3             the reference acceleration
4 s_dot_ff = self.lean_per_acc * jerk_ref
5 c_err = pos_err + (s - s_ff)                       # CoM error = axle
6             error + lean error
7 cd_err = vel_err + (s_dot - s_dot_ff)
8 xi_err = c_err + cd_err / omega                   # capture-point error
9 if integrate:
10     self.dcm_i = clamp(self.dcm_i + xi_err * dt, -g.dcm_i_max, g.
11                        dcm_i_max)
12 s_fb = -(cd_err + g.k_dcm * xi_err + g.k_dcm_i * self.dcm_i) / omega
13 s_max = g.acc_max / omega ** 2
14 s_des = clamp(s_ff + g.authority * outer_scale * s_fb, -s_max, s_max)
15 tau = tau_ff + g.k_s * (s - s_des) + g.k_sd * (s_dot - s_dot_ff)
```

4.2.2 Righe 139–153: Anteprima LIPM

Teoria (soluzione limitata del LIPM): dato il percorso pianificato delle ruote $p_{ref}(t)$, l'unica traiettoria limitata del baricentro che soddisfa $\ddot{c} = \omega^2(c - p_{ref})$ e' $c_{ref}(t) = \frac{\omega}{2} \int e^{-\omega|u|} p_{ref}(t+u) du$: il percorso filtrato con un nucleo simmetrico, quindi anticipativo. Da $c_{ref} - p_{ref}$ si ottiene l'accelerazione di riferimento $\omega^2(c_{ref} - p_{ref})$.

```

1 def smooth(self, signal, t, omega):
2     half = self.horizon_tau / omega
3     u = np.arange(-half, half + 0.5 * self.step, self.step)
4     w = np.exp(-omega * np.abs(u))
5     w /= w.sum() # normalised: a constant signal is
6     returned exactly
7     out = signal(t + u)
8     return tuple(float(w @ o) for o in out) if isinstance(out, tuple)
9     else float(w @ out)
10
11 def __call__(self, reference, t, omega):
12     c, cd = self.smooth(reference, t, omega)
13     p0, pd0 = reference(np.array([t]))
14     return omega ** 2 * (c - float(p0[0])), omega ** 2 * (cd - float(pd0[0]))

```

Listing 4.2: zmp_balance.py rr. 139-153: nucleo esponenziale simmetrico

4.3 File: controller.py (PID)

4.3.1 Righe 505–524: PD cartesiano delle gambe

Teoria (controllo in spazio operativo): $\tau = J^T F$ con $F = K_p(p_d - p) - K_d J \dot{q} + F_{ff}$. Con $K_p = \text{diag}(1500, 5000)$ N/m la rigidezza vista dai giunti e' circa 50 N·m/rad: un passo da 2 ms non puo' piu' scavalcare l'inerzia riflessa della gamba, e il ciclo limite bang-bang del PD di giunto a 160–200 N·m/rad scompare.

```

1 up_b = self._base_pose()[1].T @ np.array([0.0, 0.0, 1.0])
2 f_ff = -support * np.array([up_b[0], up_b[2]]) # share of the
3 weight, 0 in flight
4 for side, p_d in zip(('left', 'right'), targets):
5     qs = np.array([q[f'{side}_hip'], q[f'{side}_knee']])
6     qds = np.array([qd[f'{side}_hip'], qd[f'{side}_knee']])
7     p, J = self.wbr.leg_fk(*qs)
8     F = kp @ (np.asarray(p_d) - p) + kd @ (-(J @ qds)) + f_ff
9     taus.append(np.clip(J.T @ F + self.wbr.p.leg_damping * qds, -
10 LEG_TORQUE_MAX, LEG_TORQUE_MAX))

```

Listing 4.3: controller.py PID: PD cartesiano delle gambe

Capitolo 5

rotino_benchmark: Il Confronto PID-MPC

5.1 Organizzazione

Il pacchetto e' costruito per confrontare le due leggi a parita' di tutto il resto: stesso mondo, stesso URDF, stessa attuazione e stessi argomenti di scenario.

File	Ruolo
scenarios.py	catalogo degli 8 scenari: argomenti di launch, durata, metriche chiave
campaign.py	un solo scenario con PID e MPC: avvio pulito, logger, terminazione
suite.py	tutti gli scenari, poi il riepilogo della suite
logger.py	CSV a 500 Hz con gli stessi segnali per entrambe le leggi
compare.py	metriche con il loro verso, "migliore" per metrica, confronto.md e riepilogo.md
plot.py	grafici con PID (rosso) e MPC (blu) sempre sovrapposti; figura di riepilogo
zmp_analysis.py	ZMP multicorpo ricostruito e relative metriche e figure

```
ros2 run rotino_benchmark suite          # tutti gli scenari
ros2 run rotino_benchmark campaign -- spinta  # uno scenario
ros2 run rotino_benchmark compare -- <cartella> # ricostruisce tabelle e grafici
```

```
benchmark_runs/suite_<data>/
  riepilogo.md / .csv / .png      metriche chiave e bilancio PID-MPC
  <scenario>/confronto.md        tabella completa e grafici dello scenario
  <scenario>/plots/*.png         grafici sovrapposti
  <scenario>/rotino_<legge>_*.csv log, uno per legge
```

5.2 File: campaign.py

```
1 def run_one(law, scenario, duration, out_dir, log_dir):
2     kill_stale()    # nulla di una prova precedente deve pubblicare su /
3     launch = ['ros2', 'launch', f'rotino_{law}', f'rotino_{law}.launch.
4     py',
               'gui:=false', *scenario]
```

```

5     sim = subprocess.Popen(launch, ...)
6     time.sleep(2.0)                                # Gazebo in avvio
7     log = subprocess.Popen(logger, ...)             # prima del rilascio:
joint_states arriva ~1.6 s dopo odom
8     time.sleep(4.0 + duration)
9     # Terminazione ordinata: SIGINT poi SIGKILL; il log viene poi
controllato
10    # (controller non configurati = Gazebo rimasto da una prova
precedente)

```

Listing 5.1: campaign.py: ciclo di vita di un run

5.3 File: compare.py

Ogni metrica dichiara il proprio verso (minore o maggiore e' meglio, oppure solo descrittiva) e una risoluzione: sotto la risoluzione, o sotto il 5% di differenza relativa, le due leggi risultano pari.

```

1 def better(key, values):
2     direction, tie = METRIC[key][3], METRIC[key][4]
3     a, b = values.get('pid', NAN), values.get('mpc', NAN)
4     if direction == 0 or math.isnan(a) or math.isnan(b):
5         return ''
6     scale = max(abs(a), abs(b))
7     if abs(a - b) <= tie or scale < 1e-9 or abs(a - b) / scale < TIE_REL
8     :
9         return '='
10    return 'pid' if (a - b) * direction > 0 else 'mpc'

```

Listing 5.2: compare.py: chi e' migliore su una metrica

5.4 File: logger.py

```

1 def _imu_cb(self, msg):
2     self.imu = msg
3     q = msg.orientation
4     # Estrazione Roll (rotazione attorno all'asse x) dal quaternion
5     sinr_cosp = 2.0 * (q.w * q.x + q.y * q.z)
6     cosr_cosp = 1.0 - 2.0 * (q.x*q.x + q.y*q.y)
7     self.roll = math.atan2(sinr_cosp, cosr_cosp)
8     # Estrazione Yaw (rotazione attorno all'asse z) dal quaternion
9     siny_cosp = 2.0 * (q.w * q.z + q.x * q.y)
10    cosy_cosp = 1.0 - 2.0 * (q.y*q.y + q.z*q.z)
11    self.yaw = math.atan2(siny_cosp, cosy_cosp)

```

Listing 5.3: logger.py: Conversione quaternion in angoli Roll/Yaw

5.5 File: plot.py – I Grafici del Confronto

N.	File PNG	Contenuto (PID e MPC sovrapposti)
1	inseguimento.png	riferimenti e stati di θ , x , v ; errori; altezza e offset CoM desiderato
2	disturbo_e_fase.png	risposta al disturbo e ritratto di fase ($\theta, \dot{\theta}$)
3	coppie_ruote.png	coppia comune e differenziale
4	gambe.png	angoli e coppie di anca e ginocchio
5	assetto.png	rollio e imbardata
6	potenza_energia.png	potenza meccanica $\sum \tau\omega $ ed energia
7	fase_orizzontale.png	piano (e_x, e_v)
8	imu.png	accelerazioni IMU
9–12	zmp_*.png	ZMP laterale e longitudinale, vista dall'alto, impronta

```

1 for law in _laws(data):
2     d = data[law]
3     power, energy = [], [0.0]
4     for i in range(len(d['t'])):
5         p = 2.0 * (abs(d['u'][i] * d['wheel_v'][i]) + abs(d['hip_u'][i]
6 * d['hip_v'][i])
7                 + abs(d['knee_u'][i] * d['knee_v'][i]))
8         power.append(0.0 if math.isnan(p) else p)
9         if i:
10            energy.append(energy[-1] + power[-1] * (d['t'][i] - d['t'][i
11 - 1]))

```

Listing 5.4: plot.py: potenza meccanica ed energia

Capitolo 6

Conclusioni

Questo manuale ha mostrato che ogni riga del codice del progetto RoTino e' la traduzione diretta di un'equazione della teoria del controllo:

- Le **funzioni di rotazione** in `kinematics.py` implementano la geometria del gruppo $SO(3)$ tramite la formula di Rodrigues e il prodotto di rotazioni RPY.
- Il **modello VL-WIP** in `model.py` estrae automaticamente dall'URDF i parametri del modello lineare di Cui et al. (2022), garantendo coerenza tra simulazione e modello analitico.
- Il **filtro di Kalman** (`AxisKalman` in `solvers.py`) implementa le equazioni (20)-(21) del paper, fondendo IMU e odometria.
- La **cascata ZMP** in `zmp_balance.py` traduce il modello LIPM e il Capture Point in un PID: lo ZMP desiderato e' l'uscita dell'anello esterno, l'anteprima LIPM e' un filtro sul percorso pianificato.
- La **condensazione QP** in `solvers.py` trasforma un problema di programmazione dinamica a orizzonte N in un QP denso di taglia N , risolvibile in microsecondi con FISTA grazie alla struttura box-constrained.
- Il **benchmark** esegue gli stessi otto scenari con PID e MPC e produce, per ogni metrica, la legge migliore e i grafici sovrapposti.